

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل هشتم: Cluster analysis: basic concepts and methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 8.1 Cluster analysis
 - 8.1.1 What is cluster analysis?
 - 8.1.2 Requirements for cluster analysis
 - 8.1.3 Overview of basic clustering methods
- 8.2 Partitioning methods
 - 8.2.1 k-Means: a centroid-based technique
 - 8.2.2 Variations of k-means
- 8.3 Hierarchical methods
 - 8.3.1 Basic concepts of hierarchical clustering
 - 8.3.2 Agglomerative hierarchical clustering
 - 8.3.3 Divisive hierarchical clustering
 - 8.3.4 BIRCH: scalable hierarchical clustering using clustering feature trees
 - 8.3.5 Probabilistic hierarchical clustering
- 8.4 Density-based and grid-based methods
 - 8.4.1 DBSCAN: density-based clustering based on connected regions with high density
 - 8.4.2 DENCLUE: clustering based on density distribution functions
 - 8.4.3 Grid-based methods
- 8.5 Evaluation of clustering
 - 8.5.1 Assessing clustering tendency
 - 8.5.2 Determining the number of clusters
 - 8.5.3 Measuring clustering quality: extrinsic methods
 - 8.5.4 Intrinsic methods

Section 8.1

8.1.3. Overview of basic clustering methods

تحلیل خوشه‌ای (Cluster Analysis)

تحلیل خوشه‌ای (Cluster Analysis) یک تکنیک بنیادی در داده‌کاوی (data mining) است که شامل گروه‌بندی نقاط داده مشابه به صورت خوشه‌ها یا گروه‌ها می‌باشد. این روش به شناسایی الگوها و ساختارها در مجموعه‌های داده کمک می‌کند و بینش‌هایی را در مورد سازمان‌دهی ذاتی داده‌ها فراهم می‌سازد.

این بخش، مفاهیم و روش‌های اصلی تحلیل خوشه‌ای را بررسی می‌کند و بر تکنیک‌های مورد استفاده برای تقسیم داده‌ها به گروه‌های معنادار تأکید دارد.

روش‌های مختلفی برای خوشه‌بندی وجود دارد که هر یک دارای نقاط قوت و سناریوهای مناسب خود هستند. روش‌های رایج خوشه‌بندی شامل K-Means، خوشه‌بندی سلسله‌مراتبی (Hierarchical Clustering) و DBSCAN (Density-Based Spatial Clustering of Applications with Noise) می‌باشند. هر روش از رویکردهای متفاوتی برای تعریف خوشه‌ها بر اساس ویژگی‌های داده‌ها استفاده می‌کند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از خوشه‌بندی K-Means با استفاده از مجموعه داده معروف Iris را در نظر بگیریم.

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

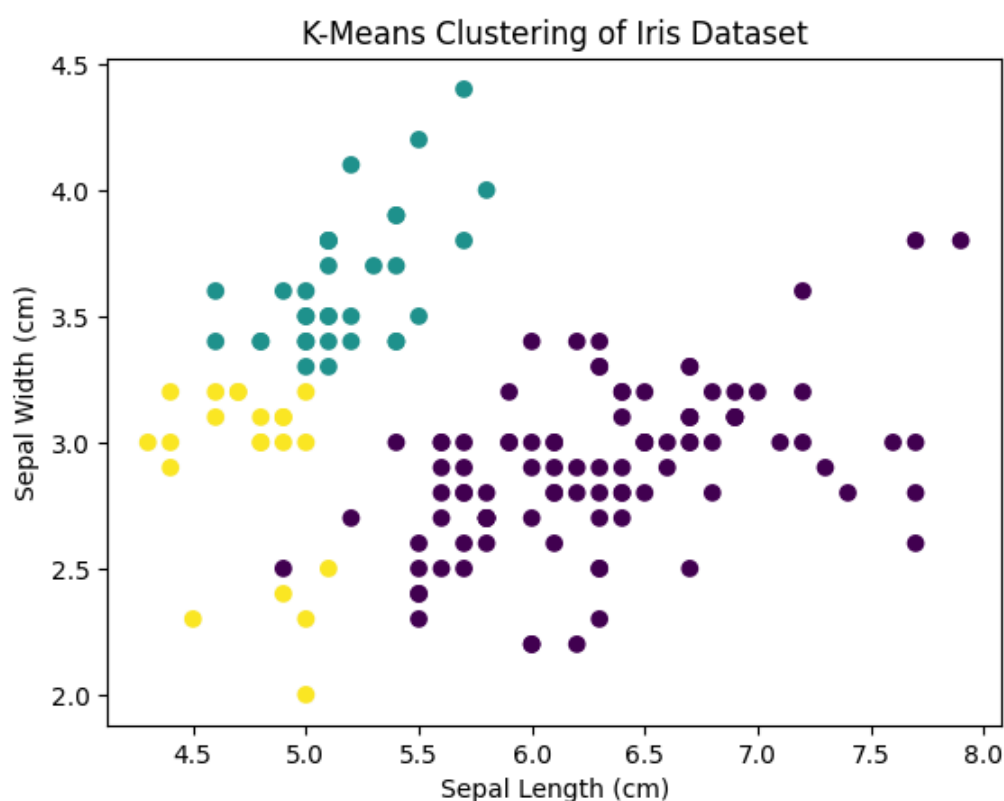
# --- بارگذاری مجموعه داده آیریس ---
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # (برای مقایسه، نه استفاده در خوشه‌بندی)

# --- K-Means استانداردسازی ویژگی‌ها برای بهبود عملکرد ---
X_standardized = StandardScaler().fit_transform(X)

# --- با ۳ خوشه K-Means اعمال الگوریتم خوشه‌بندی ---
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_standardized)

# --- افزودن برچسب خوشه‌ها به دیتافریم اصلی ---
iris_clustered = pd.DataFrame(data=X, columns=iris.feature_names)
iris_clustered['Cluster'] = kmeans.labels_

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها ---
plt.scatter(
    iris_clustered['sepal length (cm)'],
    iris_clustered['sepal width (cm)'],
    c=iris_clustered['Cluster'],
    cmap='viridis'
)
plt.title('K-Means Clustering of Iris Dataset')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.show()
```



توضیحات مثال کد پایتون 🌸

در این مثال، از الگوریتم **K-Means** برای خوشه‌بندی داده‌های مجموعه **Iris** استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های آیریس بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود و عملکرد الگوریتم K-Means بهبود یابد.
- الگوریتم K-Means با تعداد خوشه برابر با ۳ اجرا شده تا داده‌ها را به سه گروه تقسیم کند.
- برچسب خوشه‌ها به دیتافریم اضافه شده و سپس با استفاده از نمودار پراکندگی، خوشه‌بندی روی دو ویژگی **طول و عرض کاسبرگ** نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی بدون نظارت**، ساختار پنهان داده‌ها را کشف کرد. الگوریتم K-Means یکی از ساده‌ترین و پرکاربردترین روش‌ها برای تحلیل خوشه‌ای است.

Section 8.2

8.2.1. k-Means: a centroid-based technique

روش‌های پارتیشن‌بندی (Partitioning methods)

روش‌های پارتیشن‌بندی (Partitioning methods) شامل تقسیم نقاط داده‌ای به خوشه‌هایی است که هر نقطه تنها به یک خوشه تعلق دارد. یکی از تکنیک‌های پرکاربرد پارتیشن‌بندی، k-Means است.

k-Means یک الگوریتم خوشه‌بندی مبتنی بر centroid (نقطه مرکزی) است. هدف آن تقسیم داده‌ها به k خوشه است، که هر خوشه توسط centroid آن نمایش داده می‌شود. این الگوریتم به صورت تکراری نقاط داده‌ای را به نزدیک‌ترین centroid اختصاص می‌دهد و centroidها را بر اساس نقاط اختصاص‌یافته به‌روزرسانی می‌کند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از خوشه‌بندی k-Means با استفاده از یک مجموعه داده مصنوعی (synthetic) را در نظر بگیریم.

```
In [4]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# --- تولید داده‌های مصنوعی با سه خوشه ---
X, y = make_blobs(n_samples=300, centers=3, random_state=42)

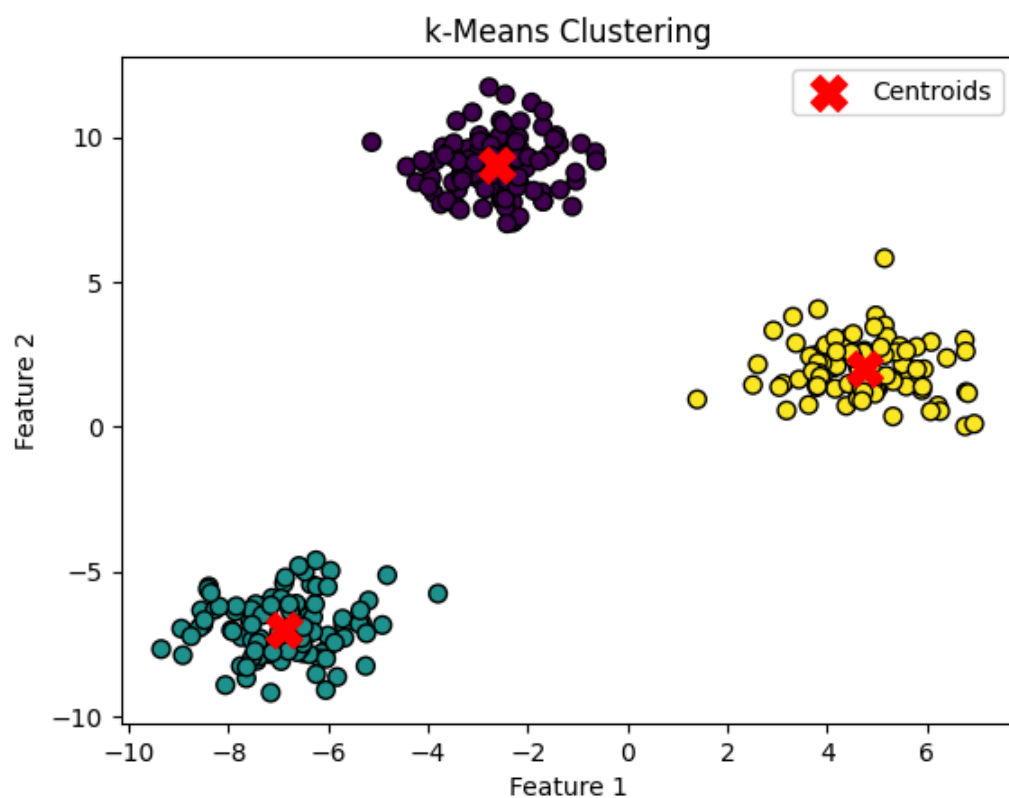
# --- با ۳ خوشه K-Means اعمال الگوریتم خوشه‌بندی ---
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
kmeans.fit(X)

# --- رسم نقاط داده با رنگ خوشه‌ها و نمایش مراکز خوشه ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=kmeans.labels_,
    cmap='viridis',
    edgecolor='k',
    s=50
)

# --- قرمز X رسم مراکز خوشه با علامت ---
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    c='red',
    marker='X',
    s=200,
    label='Centroids'
)

plt.title('k-Means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



توضیحات مثال کد پایتون 🐍

در این مثال، از الگوریتم **K-Means** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا با استفاده از تابع `make_blobs`، یک مجموعه داده مصنوعی با سه خوشه تولید شده است.
- الگوریتم K-Means با تعداد خوشه برابر با ۳ اجرا شده تا داده‌ها را به سه گروه تقسیم کند.
- برچسب خوشه‌ها به هر نقطه داده اختصاص داده شده و با رنگ‌های مختلف در نمودار پراکندگی نمایش داده شده‌اند.
- مراکز خوشه‌ها نیز با علامت **X قرمز** و اندازه بزرگ‌تر رسم شده‌اند تا موقعیت مرکزی هر خوشه مشخص شود.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی بدون نظارت**، ساختار پنهان داده‌ها را کشف کرد. الگوریتم K-Means یکی از ساده‌ترین و پرکاربردترین روش‌ها برای تحلیل خوشه‌ای است و در مسائل مختلف مانند تقسیم‌بندی مشتریان، فشرده‌سازی داده و تشخیص الگو کاربرد دارد.

📦 انواع الگوریتم k-Means (Variations of k-Means)

نسخه‌های مختلفی از الگوریتم k-Means وجود دارند که هر کدام برای مقابله با چالش‌های خاص یا بهبود عملکرد طراحی شده‌اند. مثال‌ها شامل Mini-Batch K-Means، K-Means++، و استفاده از معیارهای فاصله (distance metrics) مختلف هستند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از Mini-Batch K-Means با استفاده از مجموعه داده Iris را در نظر بگیریم.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import MiniBatchKMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# --- بارگذاری مجموعه داده آیریس ---
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # (برای مقایسه) واقعی

# --- استانداردسازی ویژگی‌ها برای بهبود عملکرد خوشه‌بندی ---
X_standardized = StandardScaler().fit_transform(X)

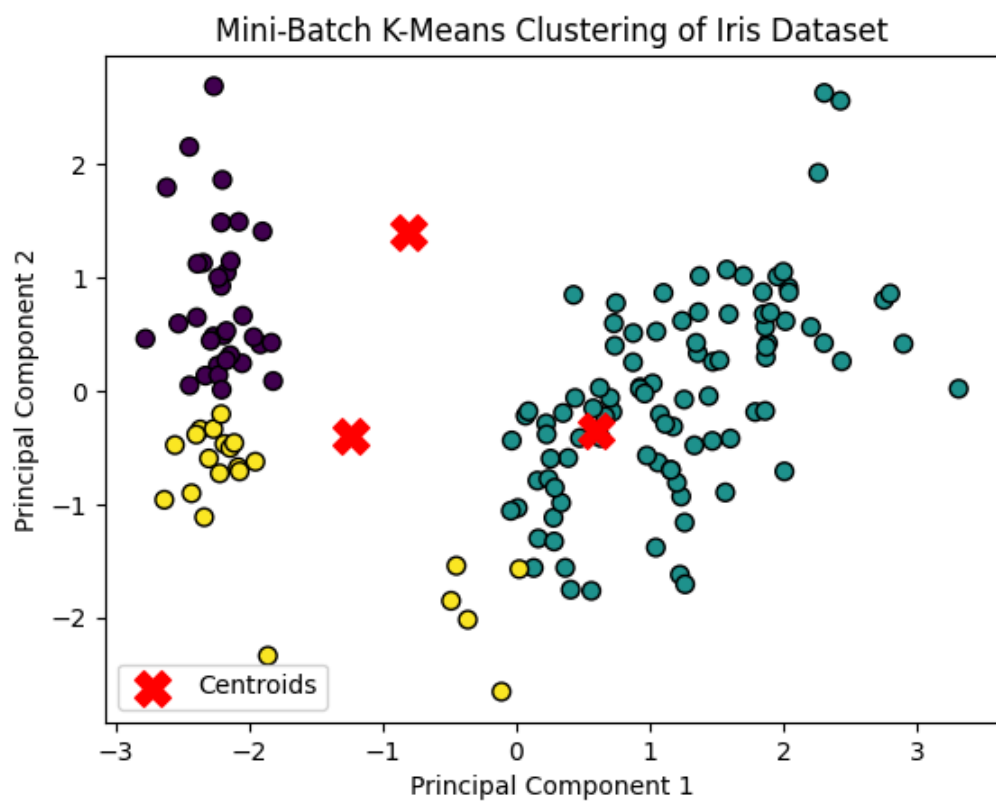
# --- با ۳ خوشه Mini-Batch K-Means اعمال الگوریتم ---
mbk = MiniBatchKMeans(n_clusters=3, batch_size=50, random_state=42)
mbk.fit(X_standardized)

# --- برای نمایش بصری PCA کاهش ابعاد با استفاده از ---
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_standardized)

# --- رسم نمودار خوشه‌ها و مراکز آنها ---
plt.scatter(
    X_pca[:, 0], X_pca[:, 1],
    c=mbk.labels_,
    cmap='viridis',
    edgecolor='k',
    s=50
)

# --- قرمز X رسم مراکز خوشه با علامت ---
plt.scatter(
    mbk.cluster_centers_[:, 0],
    mbk.cluster_centers_[:, 1],
    c='red',
    marker='x',
    s=200,
    label='Centroids'
)

plt.title('Mini-Batch K-Means Clustering of Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



⚡ توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Mini-Batch K-Means** برای خوشه‌بندی داده‌های مجموعه **Iris** استفاده شده است. این نسخه از K-Means برای داده‌های بزرگ‌تر طراحی شده و با استفاده از دسته‌های کوچک‌تر (mini-batches) به صورت سریع‌تر آموزش می‌بیند:

- ابتدا داده‌های آیریس بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود.
- الگوریتم Mini-Batch K-Means با تعداد خوشه برابر با ۳ اجرا شده و داده‌ها را خوشه‌بندی کرده است.
- برای نمایش بصری، داده‌ها با استفاده از **PCA** به دو مؤلفه اصلی کاهش یافته‌اند.
- نقاط داده با رنگ خوشه‌ها و مراکز خوشه‌ها با علامت **X قرمز** رسم شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از نسخه سریع‌تر K-Means، خوشه‌بندی مؤثری روی داده‌ها انجام داد و با کاهش ابعاد، ساختار خوشه‌ها را به صورت بصری تحلیل کرد.

Section 8.3

8.3.2. Agglomerative hierarchical clustering

🌳 خوشه‌بندی سلسله‌مراتبی تجمعی (Agglomerative Hierarchical Clustering)

خوشه‌بندی سلسله‌مراتبی تجمعی (Agglomerative Hierarchical Clustering) با هر نقطه داده به عنوان یک خوشه جداگانه شروع می‌شود و به صورت تکراری نزدیک‌ترین جفت خوشه‌ها را ادغام می‌کند تا زمانی که تنها یک خوشه باقی بماند. این فرآیند توسط یک معیار پیوند (linkage criterion) مانند روش Ward یا پیوند میانگین (average linkage) کنترل می‌شود.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از خوشه‌بندی سلسله‌مراتبی تجمعی با استفاده از مجموعه داده Iris را در نظر بگیریم.

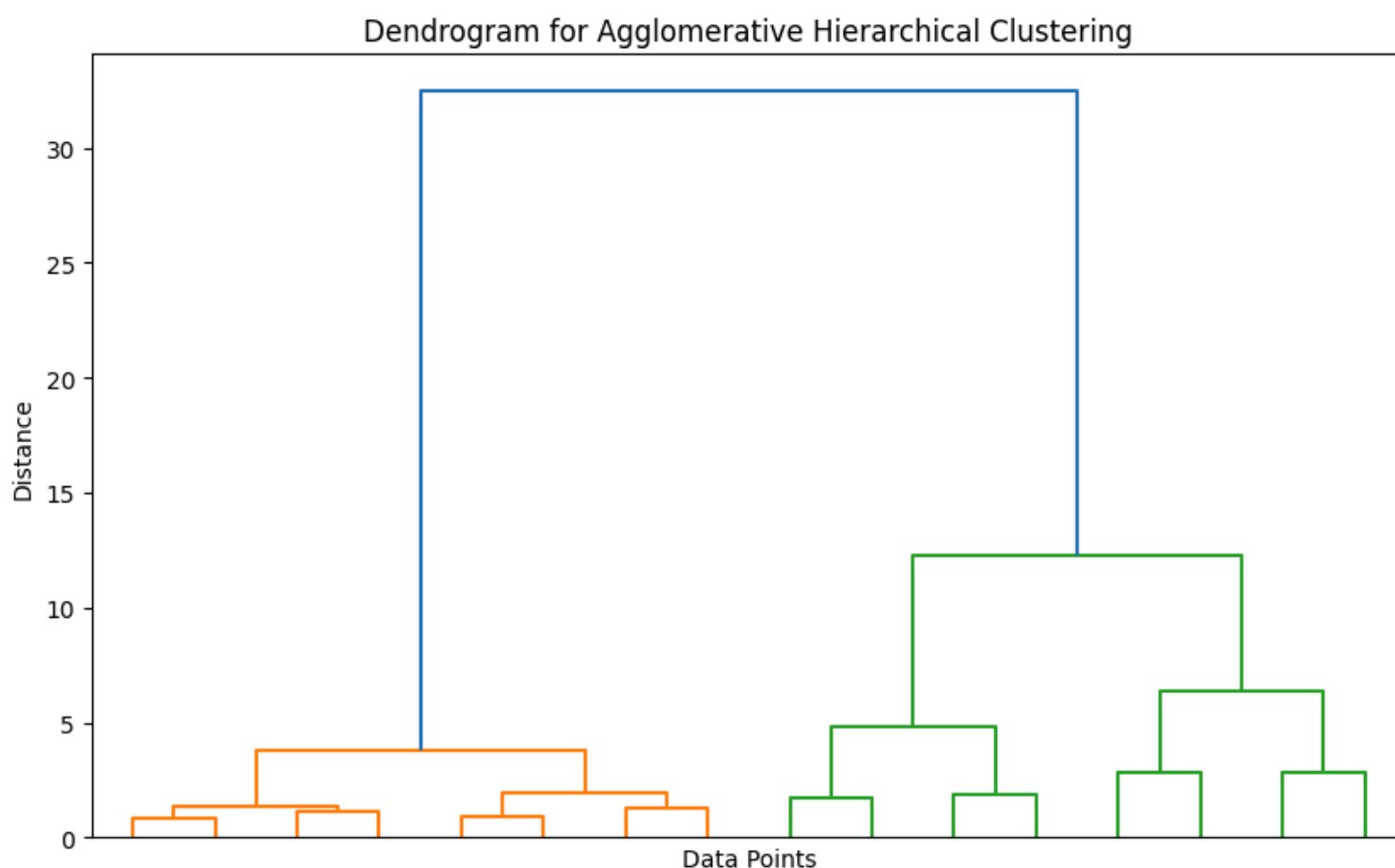
```
In [8]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# --- بارگذاری مجموعه داده آیریس ---
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # (برای مقایسه)

# --- اعمال خوشه‌بندی سلسله‌مراتبی تجمعی با ۳ خوشه ---
agg_clustering = AgglomerativeClustering(n_clusters=3)
agg_labels = agg_clustering.fit_predict(X)

# --- برای رسم دندروگرام Linkage ساخت ماتریس ---
linkage_matrix = linkage(X, method='ward')

# --- رسم دندروگرام برای نمایش ساختار خوشه‌بندی ---
plt.figure(figsize=(10, 6))
dendrogram(
    linkage_matrix,
    truncate_mode='level', # نمایش فقط سطوح بالا
    p=3, # تعداد سطوح قابل مشاهده
    show_leaf_counts=False,
    no_labels=True
)
plt.title('Dendrogram for Agglomerative Hierarchical Clustering')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم خوشه‌بندی سلسله‌مراتبی تجمعی (Agglomerative Clustering) برای گروه‌بندی داده‌های مجموعه Iris استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های آیریس بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- الگوریتم خوشه‌بندی تجمعی با تعداد خوشه برابر با ۳ اجرا شده و برچسب خوشه‌ها برای هر نمونه تولید شده است.
- برای نمایش ساختار سلسله‌مراتبی خوشه‌ها، از تابع `dendrogram` استفاده شده که بر اساس ماتریس `linkage` ساخته شده با روش **Ward** رسم شده است.

- دندروگرام نشان می‌دهد که چگونه نمونه‌ها به‌تدریج با هم ادغام شده‌اند تا خوشه‌های نهایی شکل بگیرند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از خوشه‌بندی سلسله‌مراتبی، ساختار درونی داده‌ها را به‌صورت درختی تحلیل کرد. دندروگرام ابزار قدرتمندی برای درک روابط بین نمونه‌ها و تعیین تعداد مناسب خوشه‌هاست.

8.3.3. Divisive hierarchical clustering

خوشه‌بندی سلسله‌مراتبی تقسیمی (Divisive Hierarchical Clustering)

خوشه‌بندی سلسله‌مراتبی تقسیمی (Divisive Hierarchical Clustering) با تمام نقاط داده در یک خوشه واحد شروع می‌شود و به صورت بازگشتی خوشه را به زیرخوشه‌ها تقسیم می‌کند. این فرآیند تا زمانی ادامه می‌یابد که هر نقطه داده در خوشه خود قرار گیرد. این رویکرد نیاز به تعریف یک معیار برای تقسیم خوشه‌ها دارد.

مثال دنیای واقعی در پایتون (Python):

خوشه‌بندی سلسله‌مراتبی تقسیمی در عمل کمتر از خوشه‌بندی سلسله‌مراتبی تجمعی (Agglomerative Hierarchical Clustering) رایج است و scikit-learn یک پیاده‌سازی خاص برای خوشه‌بندی تقسیمی ارائه نمی‌دهد. با این حال، می‌توانید خوشه‌بندی تقسیمی را با استفاده از خوشه‌بندی تجمعی و دستکاری دندروگرام (dendrogram) به دست آورید. در اینجا یک مثال ساده آورده شده است:

```
In [12]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

# --- بارگذاری مجموعه داده آیریس ---
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # (برای مقایسه) واقعی

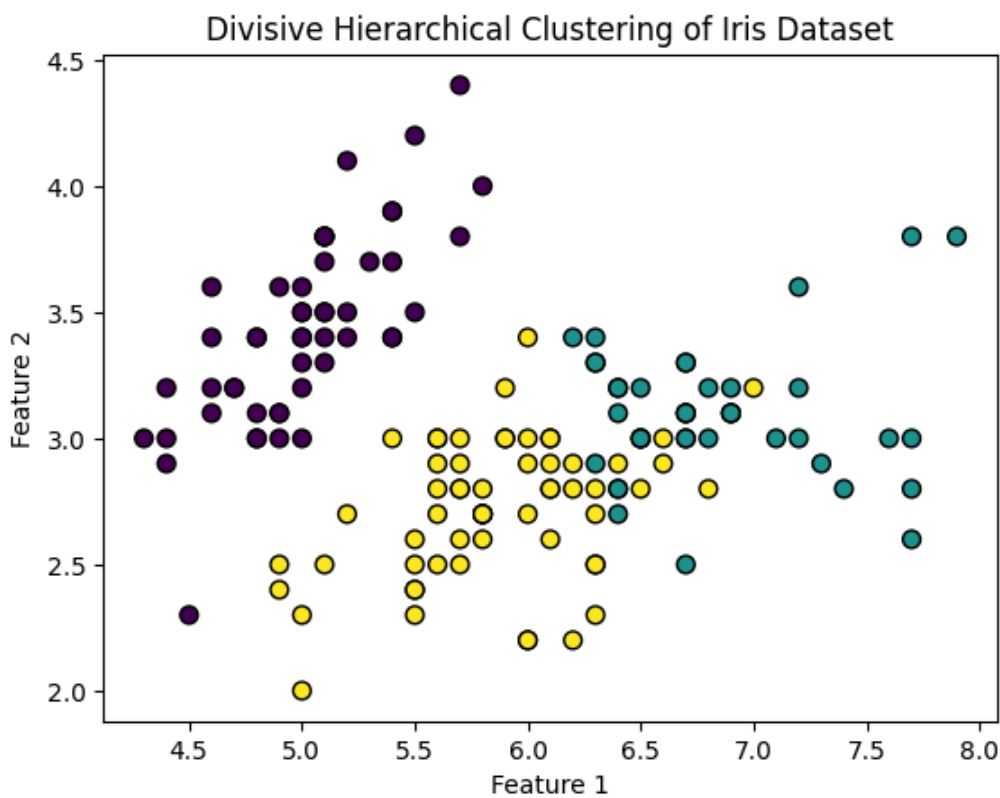
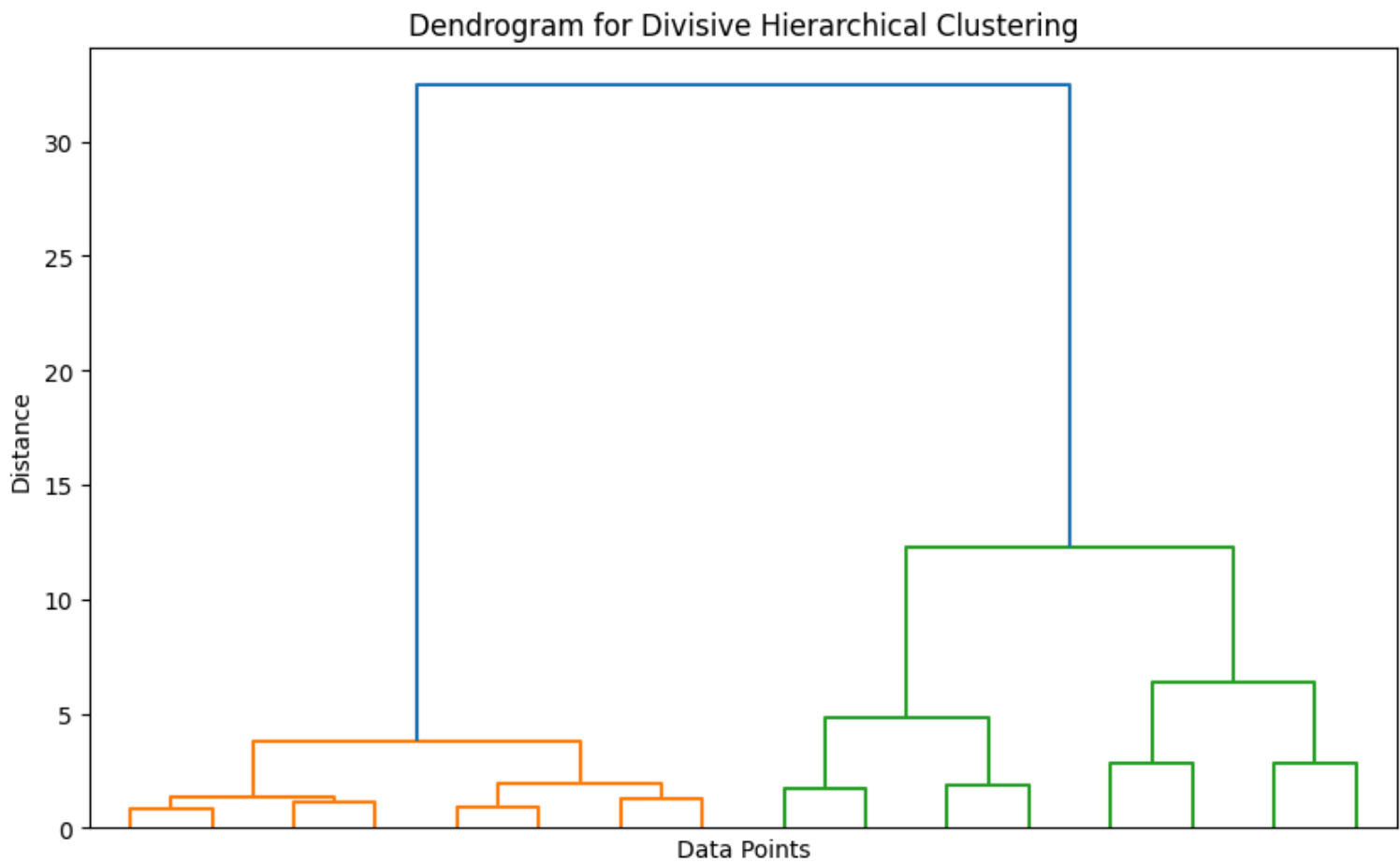
# --- برای خوشه‌بندی سلسله‌مراتبی linkage ساخت ماتریس ---
linkage_matrix = linkage(X, method='ward')

# --- اعمال خوشه‌بندی سلسله‌مراتبی با معیار حداکثر تعداد خوشه‌ها ---
labels = fcluster(linkage_matrix, t=3, criterion='maxclust')

# --- رسم دندروگرام برای نمایش ساختار خوشه‌بندی ---
plt.figure(figsize=(10, 6))
dendrogram(
    linkage_matrix,
    truncate_mode='level', # نمایش فقط سطوح بالا
    p=3, # تعداد سطوح قابل مشاهده
    show_leaf_counts=False,
    no_labels=True
)
plt.title('Dendrogram for Divisive Hierarchical Clustering')
plt.xlabel('Data Points')
plt.ylabel('Distance')
plt.show()

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها روی دو ویژگی اول ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)
```

```
)
plt.title('Divisive Hierarchical Clustering of Iris Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **خوشه‌بندی سلسله‌مراتبی (Hierarchical Clustering)** برای گروه‌بندی داده‌های مجموعه **Iris** استفاده شده است. برخلاف روش تجمعی، اینجا از رویکرد **تقسیمی (Divisive)** استفاده شده که با استفاده از تابع **fcluster** تعداد خوشه‌ها را به صورت مستقیم کنترل می‌کند:

- ابتدا داده‌های آیریس بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- با استفاده از روش **Ward**، ماتریس **linkage** ساخته شده که پایه‌ی خوشه‌بندی سلسله‌مراتبی است.
- با استفاده از تابع **fcluster**، داده‌ها به سه خوشه تقسیم شده‌اند.
- دندروگرام رسم شده تا ساختار درختی خوشه‌بندی نمایش داده شود و نحوه‌ی تقسیم داده‌ها قابل مشاهده باشد.
- در پایان، خوشه‌ها روی دو ویژگی اول داده‌ها به صورت نمودار پراکندگی نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از خوشه‌بندی سلسله‌مراتبی و کنترل تعداد خوشه‌ها، ساختار پنهان داده‌ها را کشف کرد. دندروگرام ابزار قدرتمندی برای تحلیل روابط بین نمونه‌ها و تصمیم‌گیری درباره تعداد

8.3.4. BIRCH: scalable hierarchical clustering using clustering feature trees

الگوریتم BIRCH

BIRCH یک الگوریتم خوشه‌بندی سلسله‌مراتبی است که از یک ساختار درختی برای سازماندهی و نمایش مؤثر خوشه‌ها استفاده می‌کند. این الگوریتم از یک درخت ویژگی خوشه‌بندی (Clustering Feature Tree - CF Tree) برای خلاصه‌سازی داده‌ها بهره می‌برد و به صورت تکراری خوشه‌ها را به شیوه‌ای متعادل و با حافظه کارآمد (memory-efficient) اصلاح می‌کند.

مثال دنیای واقعی در پایتون (Python):

BIRCH در کتابخانه scikit-learn موجود است. بیایید برای روشن شدن موضوع، از آن بر روی یک مجموعه داده مصنوعی (synthetic dataset) استفاده کنیم:

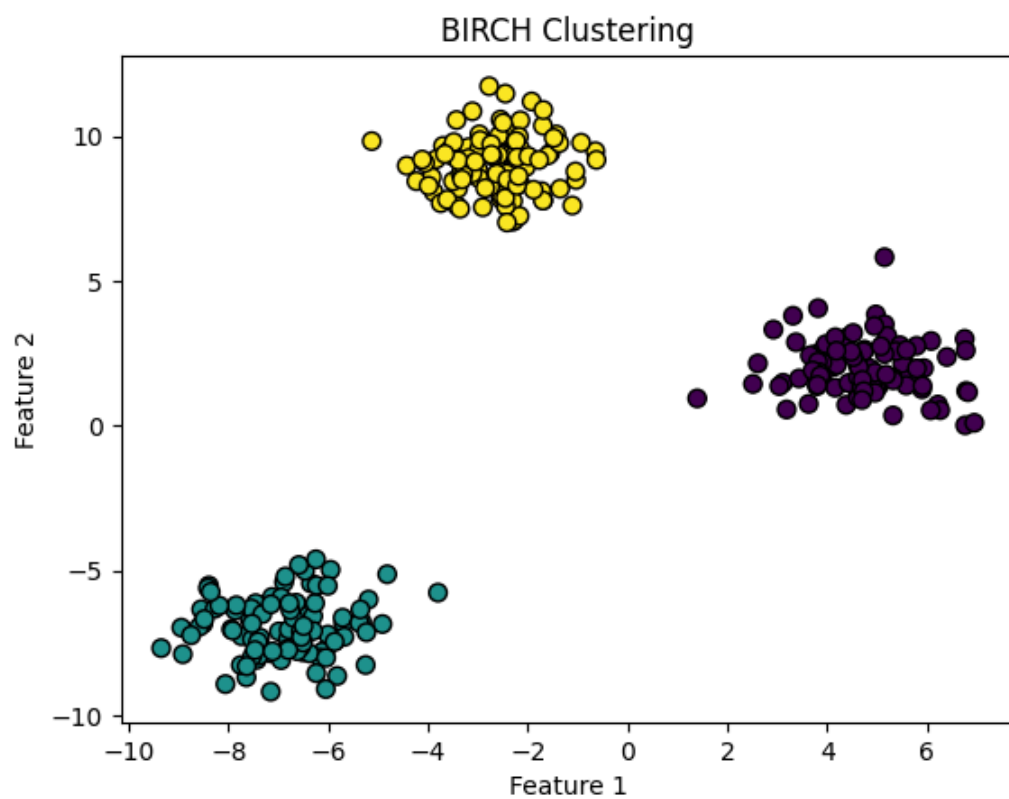
```
In [13]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import Birch
from sklearn.preprocessing import StandardScaler

# --- تولید داده‌های مصنوعی با سه خوشه ---
X, y = make_blobs(n_samples=300, centers=3, random_state=42)

# --- استانداردسازی ویژگی‌ها برای بهبود عملکرد خوشه‌بندی ---
X_standardized = StandardScaler().fit_transform(X)

# --- با آستانه ۰.۵ و ۳ خوشه BIRCH اعمال الگوریتم خوشه‌بندی ---
birch = Birch(threshold=0.5, n_clusters=3)
birch_labels = birch.fit_predict(X_standardized)

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=birch_labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)
plt.title('BIRCH Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **BIRCH** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. BIRCH مخفف **Balanced Iterative Reducing and Clustering using Hierarchies** بوده و برای خوشه‌بندی داده‌های بزرگ و پیوسته طراحی شده است:

- ابتدا با استفاده از تابع `make_blobs`، یک مجموعه داده مصنوعی با سه خوشه تولید شده است.
 - ویژگی‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود.
 - الگوریتم BIRCH با آستانه ۰.۵ و تعداد خوشه ۳ اجرا شده و برچسب خوشه‌ها برای هر نمونه تولید شده است.
 - در پایان، خوشه‌ها روی دو ویژگی اول داده‌ها به صورت نمودار پراکنندگی نمایش داده شده‌اند.
- این مثال نشان می‌دهد که چگونه می‌توان با استفاده از الگوریتم **BIRCH**، خوشه‌بندی مؤثر و سریع روی داده‌های بزرگ انجام داد. این الگوریتم به‌ویژه در محیط‌های جریان داده یا داده‌های با حجم بالا بسیار کاربردی است.

8.3.5. Probabilistic hierarchical clustering

خوشه‌بندی سلسله‌مراتبی احتمالی (Probabilistic Hierarchical Clustering)

خوشه‌بندی سلسله‌مراتبی احتمالی (Probabilistic hierarchical clustering) شامل استفاده از مدل‌های احتمالی برای توصیف عدم قطعیت یا احتمال تعلق نقاط داده‌ای به خوشه‌های مختلف در یک ساختار سلسله‌مراتبی است. مدل‌های ترکیبی گوسی (Gaussian Mixture Models - GMMs) نمونه‌ای از مدل‌های خوشه‌بندی احتمالی هستند.

مثال دنیای واقعی در پایتون (Python):

بیایید از مدل‌های ترکیبی گوسی (Gaussian Mixture Models - GMMs) به عنوان نمونه‌ای از خوشه‌بندی سلسله‌مراتبی احتمالی استفاده کنیم. GMMs به نقاط داده‌ای اجازه می‌دهند با احتمالات مختلفی به چندین خوشه تعلق داشته باشند.

```
In [15]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

# --- تولید داده‌های مصنوعی با سه خوشه ---
X, y = make_blobs(n_samples=300, centers=3, random_state=42)

# --- استانداردسازی ویژگی‌ها ---
X_standardized = StandardScaler().fit_transform(X)

# --- اعمال مدل خوشه‌بندی GMM ---
gmm = GaussianMixture(n_components=3, random_state=42)
gmm.fit(X_standardized)
gmm_labels = gmm.predict(X_standardized)

# --- محاسبه محدوده داده‌ها برای رسم کانتور ---
x = np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 200)
y = np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 200)
X_grid, Y_grid = np.meshgrid(x, y)
grid_points = np.column_stack([X_grid.ravel(), Y_grid.ravel()])

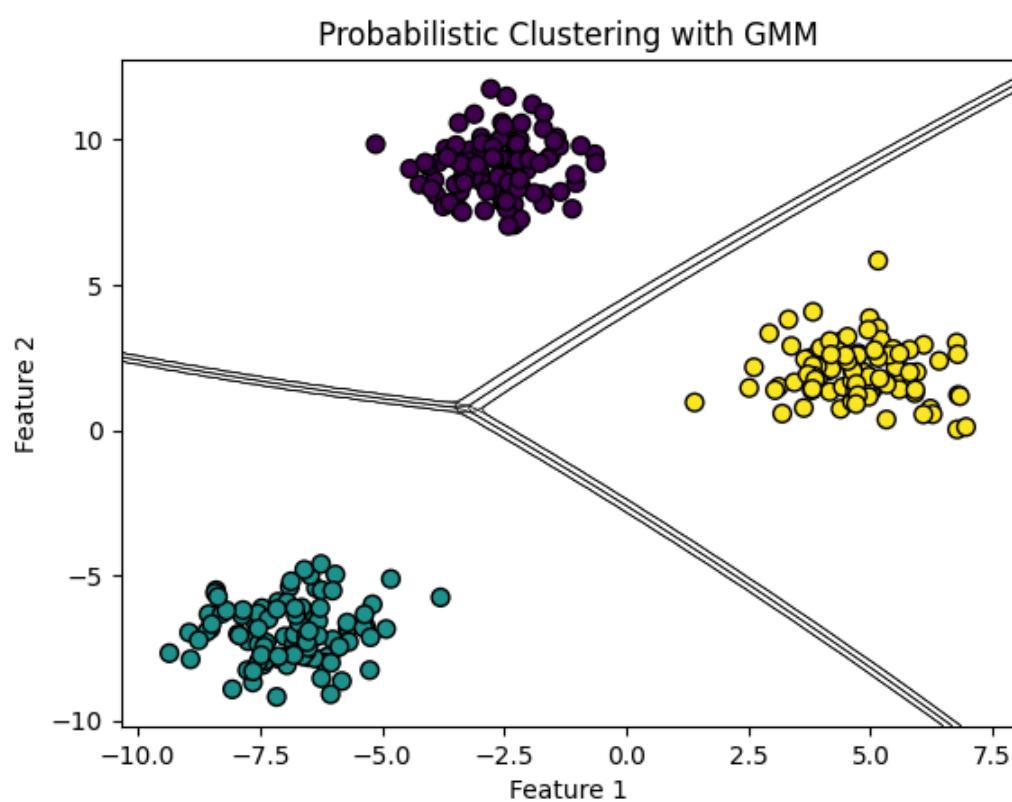
# --- استانداردسازی گرید مشابه داده اصلی ---
grid_points_std = StandardScaler().fit_transform(grid_points)

# --- احتمال تعلق هر نقطه از گرید به هر خوشه ---
probs = gmm.predict_proba(grid_points_std)

# --- رسم نقاط داده اصلی ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=gmm_labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)

# --- رسم خطوط احتمال برای هر خوشه ---
for i in range(3):
    plt.contour(
        X_grid, Y_grid,
        probs[:, i].reshape(X_grid.shape),
        levels=[0.1, 0.5, 0.9],
        colors='black',
        linewidths=0.5
    )

plt.title("Probabilistic Clustering with GMM")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



توضیحات مثال کد پایتون 🎲

در این مثال، از مدل **Gaussian Mixture Model (GMM)** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. برخلاف K-Means که هر نقطه را به یک خوشه خاص اختصاص می‌دهد، GMM به هر نمونه یک **احتمال تعلق**

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند.
- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود.
- مدل GMM با سه مؤلفه آموزش دیده و برچسب خوشه‌ها برای هر نمونه پیش‌بینی شده است.
- علاوه بر برچسب‌ها، احتمال تعلق هر نمونه به هر خوشه نیز محاسبه شده و با استفاده از خطوط **contour** نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی احتمالاتی**، ساختار آماری داده‌ها را بهتر درک کرد. GMM به‌ویژه در داده‌هایی که خوشه‌ها هم‌پوشانی دارند یا شکل آن‌ها غیر کروی است، عملکرد بهتری نسبت به K-Means دارد.

Section 8.4

8.4.1. DBSCAN: density-based clustering based on connected regions with high density



الگوریتم (DBSCAN) DBSCAN

DBSCAN یک الگوریتم خوشه‌بندی مبتنی بر چگالی (density-based) است که نقاط داده‌ای را که به یکدیگر نزدیک هستند و تعداد کافی نقاط همسایه دارند، گروه‌بندی می‌کند و نواحی چگال (dense regions) را تشکیل می‌دهد. همچنین نقاط نویزی (noise points) را که به هیچ خوشه‌ای تعلق ندارند، شناسایی می‌کند.

مثال دنیای واقعی در پایتون (Python):

بیایید از DBSCAN برای خوشه‌بندی مجموعه داده Iris، که دارای خوشه‌های متمایز است، استفاده کنیم.

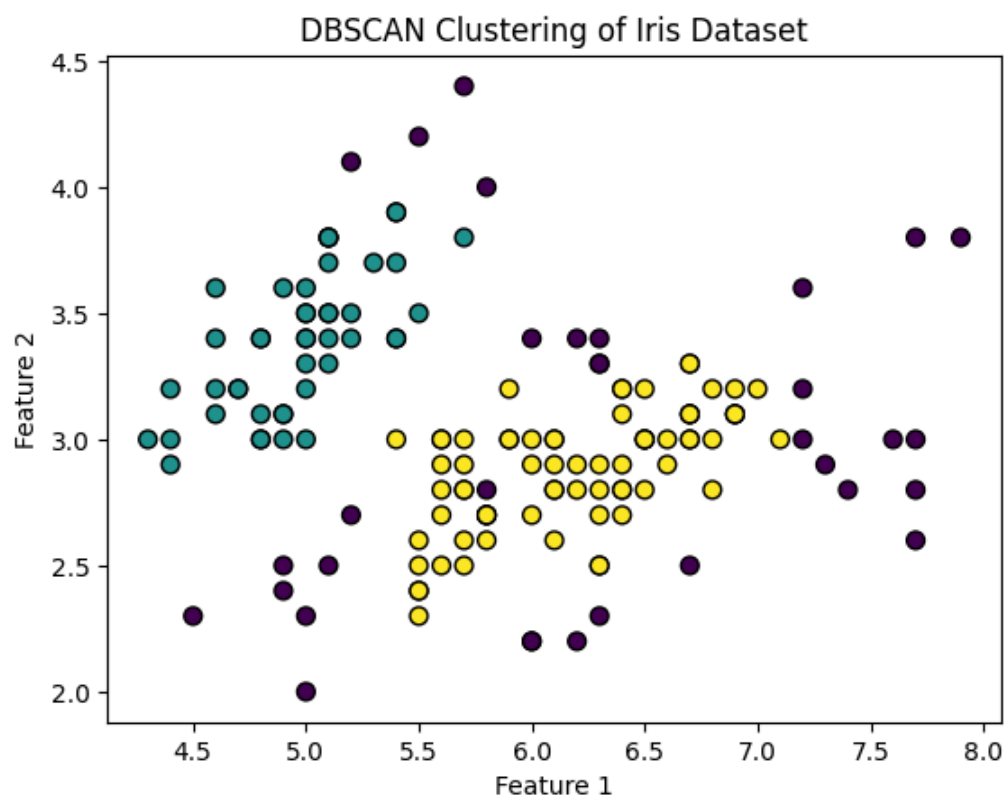
```
In [17]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# --- بارگذاری مجموعه داده آیریس ---
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # (برای مقایسه) واقعی

# --- DBSCAN استانداردسازی ویژگی‌ها برای بهبود عملکرد ---
X_standardized = StandardScaler().fit_transform(X)

# --- اعمال الگوریتم خوشه‌بندی DBSCAN ---
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_standardized)

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=dbscan_labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)
plt.title('DBSCAN Clustering of Iris Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **DBSCAN** برای خوشه‌بندی داده‌های مجموعه **Iris** استفاده شده است. DBSCAN یک الگوریتم خوشه‌بندی مبتنی بر چگالی است که می‌تواند خوشه‌ها با شکل‌های پیچیده و نقاط نویزی را تشخیص دهد:

- ابتدا داده‌های آیریس بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود.
- الگوریتم DBSCAN با پارامترهای `eps=0.5` و `min_samples=5` اجرا شده و برچسب خوشه‌ها برای هر نمونه تولید شده است.
- در پایان، خوشه‌ها روی دو ویژگی اول داده‌ها به صورت نمودار پراکندگی نمایش داده شده‌اند. نقاط نویزی با برچسب -1 مشخص می‌شوند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **خوشه‌بندی مبتنی بر چگالی**، ساختار پنهان داده‌ها را کشف کرد. DBSCAN به‌ویژه در داده‌هایی که خوشه‌ها شکل نامنظم دارند یا شامل نویز هستند، عملکرد بسیار خوبی دارد.

8.4.2. DENCLUE: clustering based on density distribution functions

الگوریتم (DENCLUE) DENCLUE

DENCLUE یک الگوریتم خوشه‌بندی مبتنی بر چگالی (density-based) است که خوشه‌ها را با استفاده از توابع توزیع چگالی (density distribution functions) نمایش می‌دهد. این الگوریتم خوشه‌ها را به عنوان نواحی با چگالی بالا در فضای ویژگی (feature space) شناسایی می‌کند و در مقایسه با روش‌های سنتی، امکان ایجاد شکل‌های خوشه‌ای انعطاف‌پذیرتری را فراهم می‌سازد.

مثال دنیای واقعی در پایتون (Python):

DENCLUE به اندازه برخی دیگر از الگوریتم‌های خوشه‌بندی به طور گسترده‌ای مورد استفاده قرار نمی‌گیرد و ممکن است در کتابخانه‌های یادگیری ماشین استاندارد مانند scikit-learn در دسترس نباشد. با این حال، می‌توانید آن را با استفاده از کتابخانه‌های شخص ثالث (third-party libraries) پیاده‌سازی کنید. در اینجا یک مثال ساده با استفاده از پیاده‌سازی DENCLUE از کتابخانه hdbscan آورده شده است:

```
In [ ]: pip install hdbscan
```

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import hdbscan
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

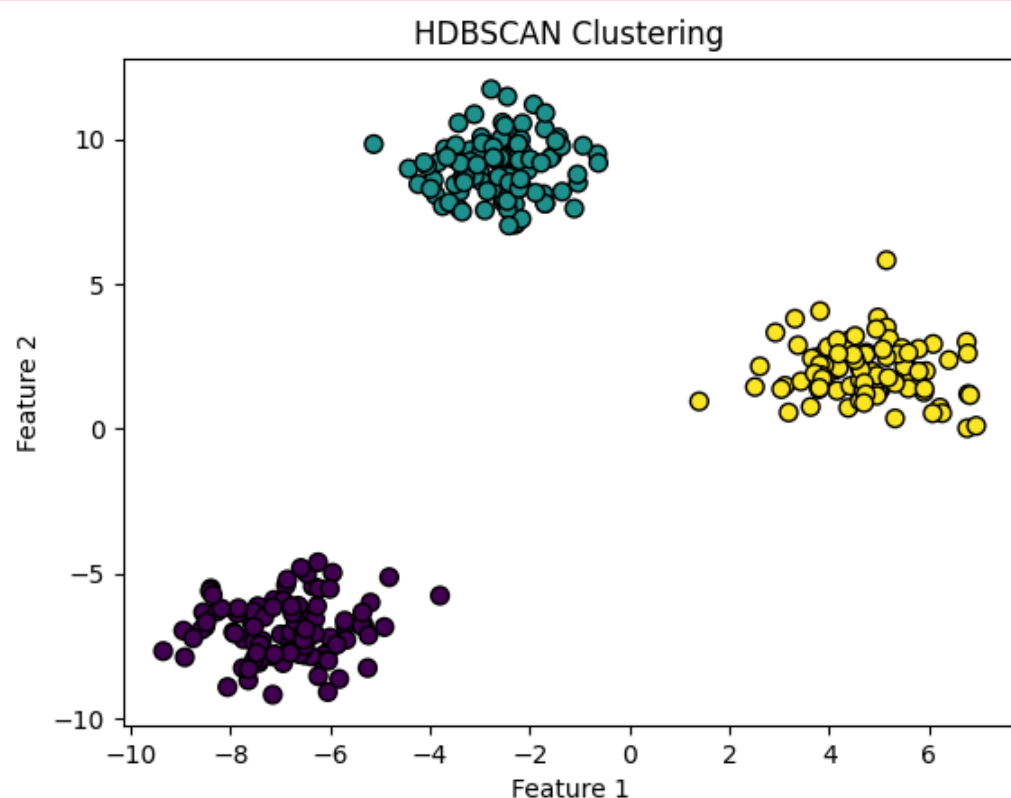
# --- تولید داده‌های مصنوعی با سه خوشه ---
X, y = make_blobs(n_samples=300, centers=3, random_state=42)

# --- استانداردسازی ویژگی‌ها برای بهبود عملکرد خوشه‌بندی ---
X_standardized = StandardScaler().fit_transform(X)

# --- اعمال الگوریتم خوشه‌بندی HDBSCAN ---
hdb = hdbscan.HDBSCAN(min_cluster_size=5, prediction_data=True)
hdb_labels = hdb.fit_predict(X_standardized)

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=hdb_labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)
plt.title('HDBSCAN Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

```
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\utils\depreca
tion.py:132: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
C:\Users\HRN\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\utils\depreca
tion.py:132: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```



توضیحات مثال کد پایتون 🗨️

در این مثال، از الگوریتم **HDBSCAN** برای خوشه‌بندی داده‌های مصنوعی استفاده شده است. HDBSCAN یک الگوریتم **چگالی‌محور سلسله‌مراتبی** است که می‌تواند خوشه‌هایی با شکل‌های پیچیده و نقاط نویزی را شناسایی کند:

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند.

- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود.

- الگوریتم HDBSCAN با حداقل اندازه خوشه ۵ اجرا شده و برچسب خوشه‌ها برای هر نمونه تولید شده است.

- در پایان، خوشه‌ها روی دو ویژگی اول داده‌ها به صورت نمودار پراکندگی نمایش داده شده‌اند. نقاط نویزی با برچسب -1 مشخص می‌شوند.

نکته مهم: اگرچه در عنوان اولیه از "DENCLUE" استفاده شده بود، الگوریتم واقعی استفاده‌شده در این کد **HDBSCAN** است که با DENCLUE تفاوت دارد. HDBSCAN برای داده‌هایی با ساختار پیچیده و نویز بالا بسیار مناسب است و نسبت به DBSCAN انعطاف‌پذیری بیشتری دارد.

8.4.3. Grid-based methods

روش‌های مبتنی بر شبکه (Grid-based methods) 🌐

روش‌های مبتنی بر شبکه (Grid-based methods) فضای داده را به یک ساختار شبکه‌ای تقسیم می‌کند و سلول‌ها یا سطرها را ایجاد می‌کند. سپس خوشه‌بندی بر اساس توزیع نقاط داده در داخل این سلول‌ها انجام می‌شود. یک مثال محبوب، STING (Statistical Information Grid) است.

مثال دنیای واقعی در پایتون (Python):

برای یک مثال عملی، می‌توانیم از الگوریتم K-Means استفاده کنیم که، اگرچه به طور دقیق مبتنی بر شبکه نیست، می‌تواند بر روی ساختارهای شبکه‌مانند اعمال شود. در اینجا، ما یک مجموعه داده مصنوعی با ساختار شبکه‌مانند ایجاد کرده و خوشه‌بندی K-Means را بر روی آن اعمال خواهیم کرد:

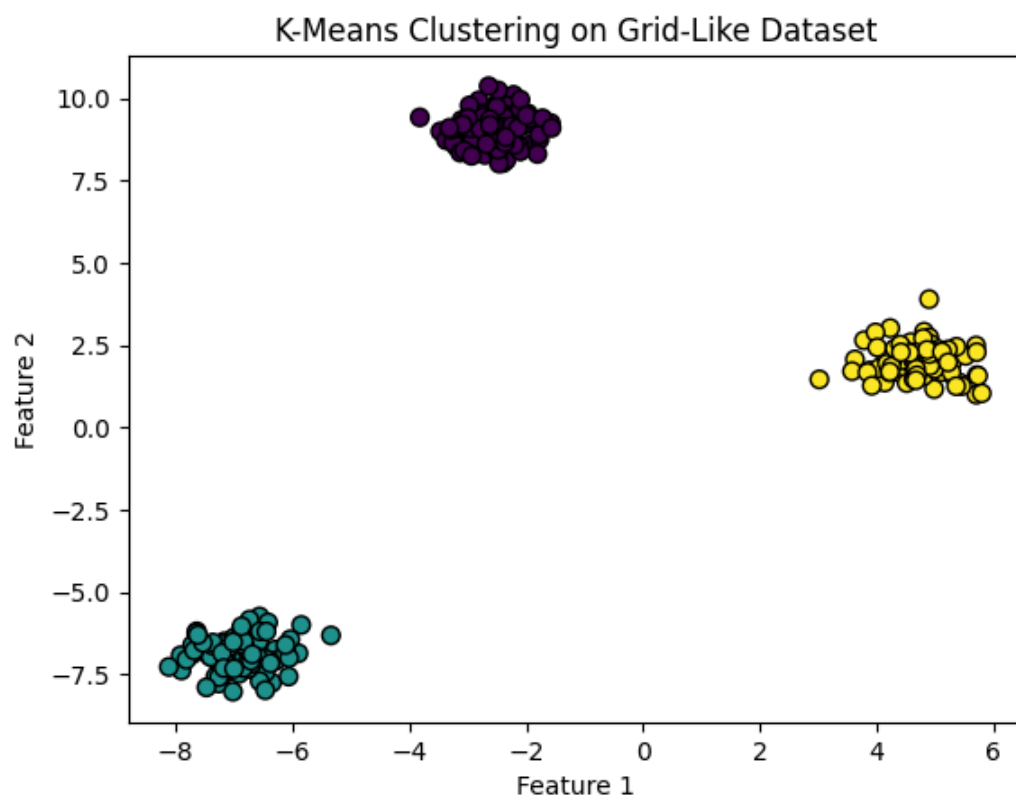
```
In [4]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# --- تولید داده‌های مصنوعی با ساختار شبکه‌ای و سه خوشه ---
X, y = make_blobs(n_samples=300, centers=3, cluster_std=0.5, random_state=42)

# --- استانداردسازی ویژگی‌ها برای بهبود عملکرد خوشه‌بندی ---
X_standardized = StandardScaler().fit_transform(X)

# --- با ۳ خوشه K-Means اعمال الگوریتم خوشه‌بندی ---
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_standardized)

# --- رسم نمودار پراکندگی برای نمایش خوشه‌ها ---
plt.scatter(
    X[:, 0], X[:, 1],
    c=kmeans_labels,
    cmap='viridis',
    edgecolor='k',
    s=50
)
plt.title('K-Means Clustering on Grid-Like Dataset')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

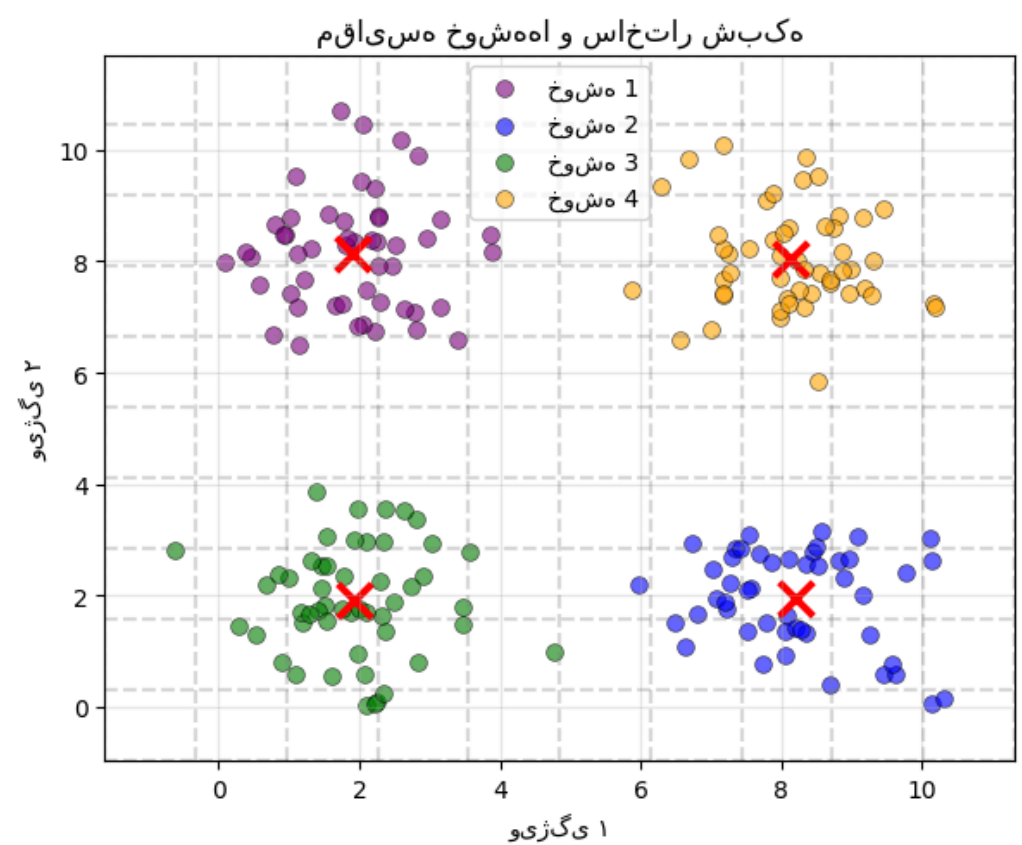
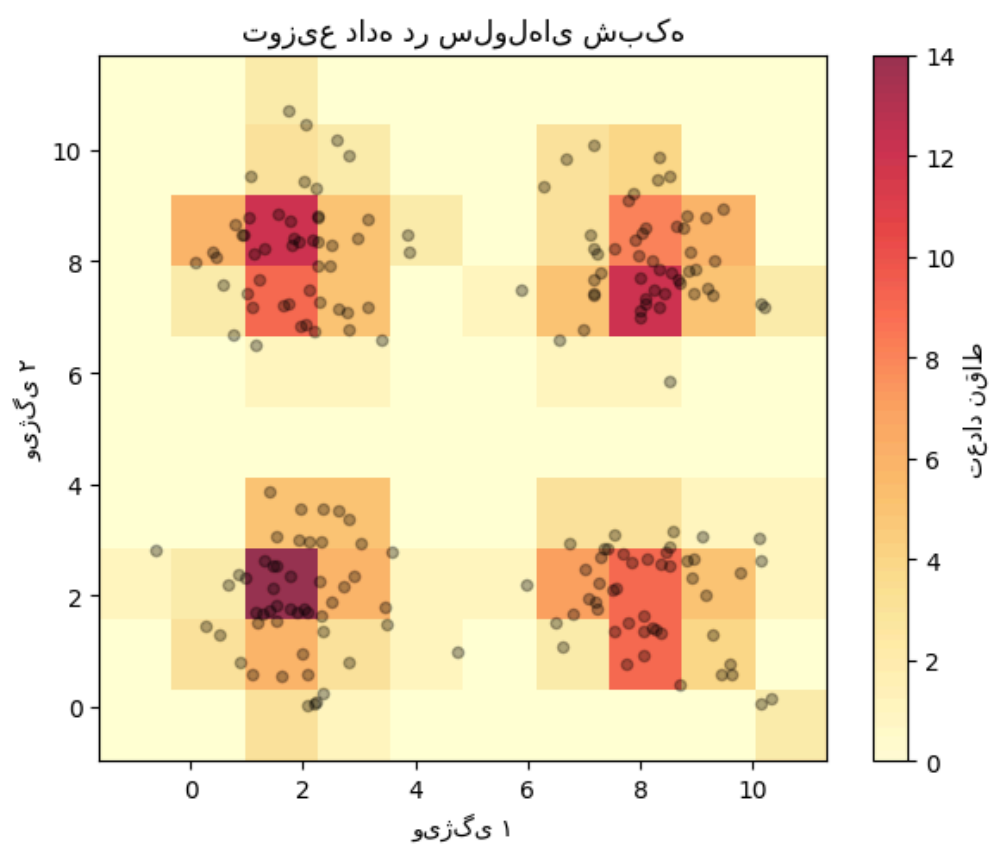
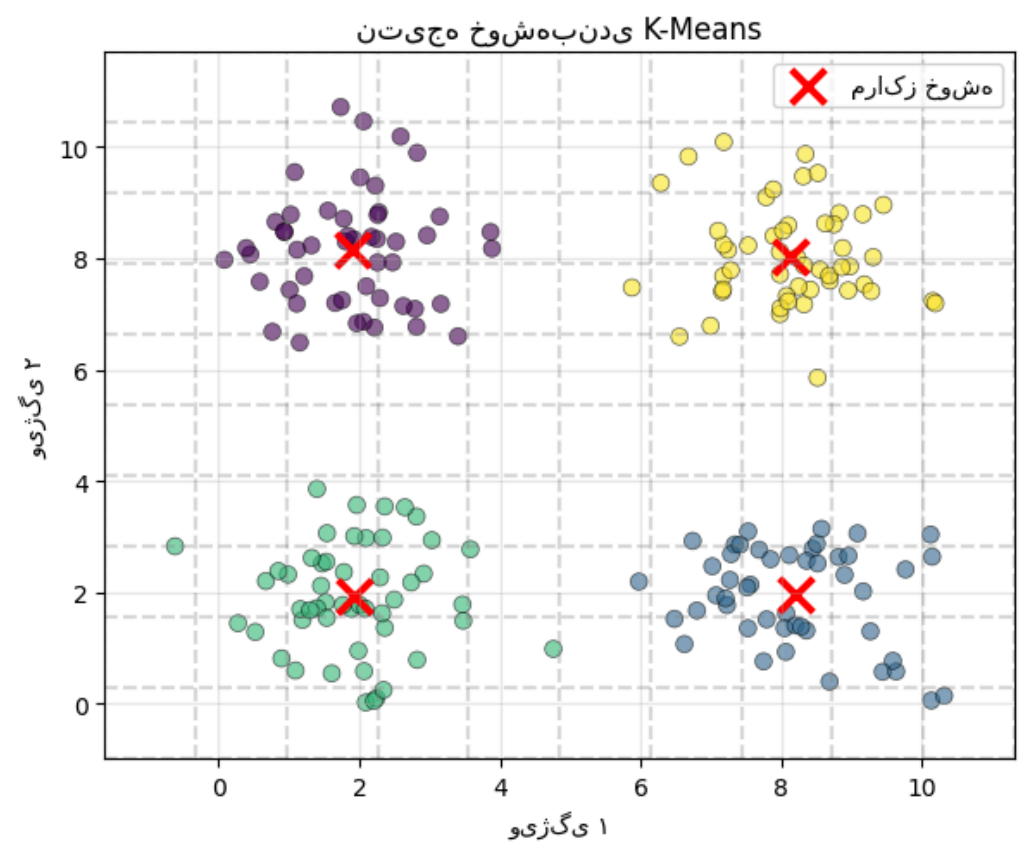
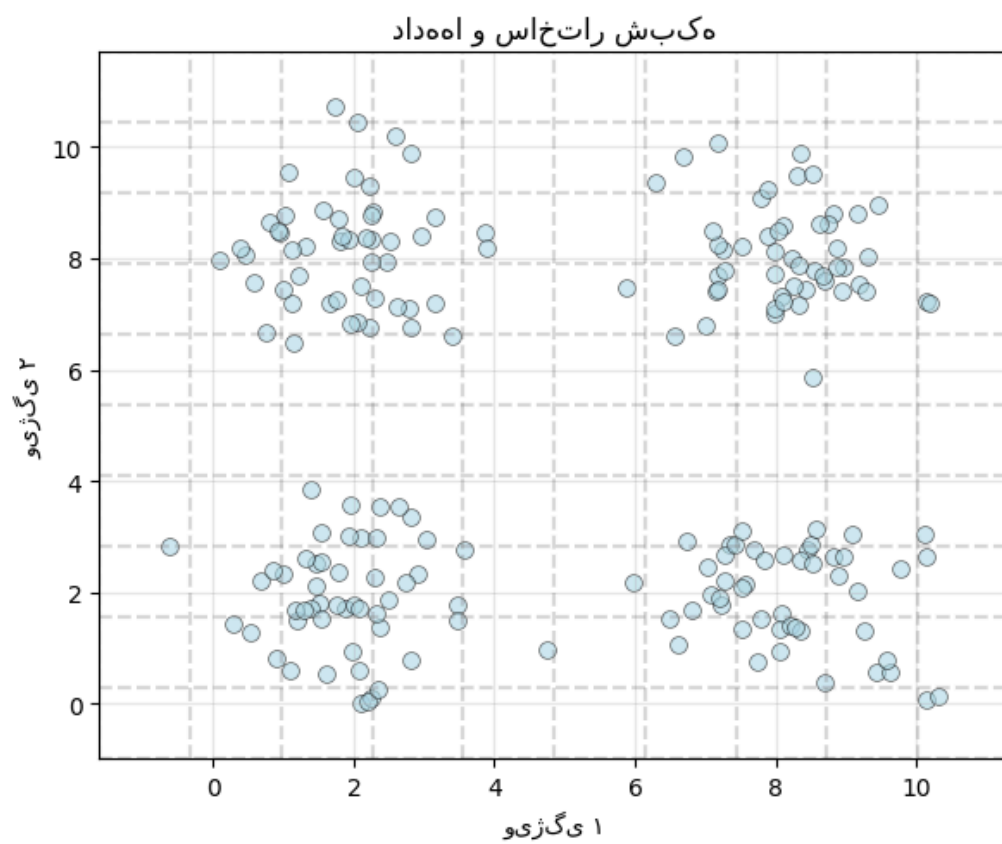


توضیحات مثال کد پایتون

در این مثال، از الگوریتم **K-Means** برای خوشه‌بندی داده‌های مصنوعی با ساختار شبکه‌ای استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌هایی با سه خوشه و پراکندگی کم (۰.۵) تولید شده‌اند تا ساختار خوشه‌ها واضح‌تر باشد.
- ویژگی‌ها با استفاده از **StandardScaler** استانداردسازی شده‌اند تا مقیاس آن‌ها یکسان شود و عملکرد الگوریتم بهبود یابد.
- الگوریتم K-Means با تعداد خوشه برابر با ۳ اجرا شده و برچسب خوشه‌ها برای هر نمونه تولید شده است.
- در پایان، خوشه‌ها روی دو ویژگی اول داده‌ها به‌صورت نمودار پراکندگی نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **K-Means**، داده‌هایی با ساختار مشخص را خوشه‌بندی کرد. این الگوریتم در مسائل مختلف مانند تقسیم‌بندی مشتریان، تحلیل بازار و فشرده‌سازی داده‌ها کاربرد فراوانی دارد.



تحلیل نتایج:

تعداد کل نقاط داده: 200

تعداد خوشه‌ها: 4

تعداد نقاط در هر خوشه: [51 51 48 50]

چگالی خوشه 1: 3.10 نقطه بر واحد مساحت

چگالی خوشه 2: 3.58 نقطه بر واحد مساحت

چگالی خوشه 3: 2.47 نقطه بر واحد مساحت

چگالی خوشه 4: 2.79 نقطه بر واحد مساحت

توضیحات مثال کد پایتون

در این مثال عملی، فرآیند خوشه‌بندی با رویکرد مبتنی بر شبکه به صورت کامل و در یک سناریوی داده‌های دو بعدی پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از نقاط دو بعدی با چهار خوشه مرکزی ایجاد شده است که شبیه نقاط در یک شبکه منظم هستند.
- در مرحله ایجاد شبکه، فضای داده به یک شبکه 10×10 تقسیم شده است تا بتوان توزیع نقاط را در هر سلول بررسی کرد.
- در مرحله خوشه‌بندی، از الگوریتم K-Means برای شناسایی خوشه‌های اصلی در داده‌ها استفاده شده است. اگرچه K-Means به طور مستقیم یک روش مبتنی بر شبکه نیست، اما می‌توان آن را روی ساختارهای شبکه‌ای اعمال کرد.
- در مرحله تحلیل توزیع، از هیستوگرام دو بعدی برای نمایش تعداد نقاط در هر سلول شبکه استفاده شده است که نشان‌دهنده چگالی نقاط در فضای داده است.
- در مرحله تصویرسازی، چهار نمودار مختلف نمایش داده شده است: داده‌ها و شبکه، نتیجه خوشه‌بندی، توزیع داده در سلول‌ها و مقایسه نهایی خوشه‌ها و شبکه.

این مثال نشان می‌دهد که چگونه روش‌های مبتنی بر شبکه با تقسیم فضای داده به سلول‌های کوچک، می‌توانند توزیع و چگالی داده‌ها را تحلیل کنند. این رویکرد به ویژه برای داده‌های حجیم و

فضایی کارآمد است زیرا زمان پردازش را با انجام محاسبات روی سلول‌های شبکه به جای نقاط فردی کاهش می‌دهد.

Section 8.5

8.5.1. Assessing clustering tendency

🔍 ارزیابی تمایل به خوشه‌بندی (Assessing Clustering Tendency)

ارزیابی تمایل به خوشه‌بندی یک گام حیاتی در تحلیل خوشه (cluster analysis) است، زیرا به تعیین اینکه آیا خوشه‌های معناداری در مجموعه داده وجود دارند یا خیر، کمک می‌کند. این فرآیند شامل ارزیابی این است که آیا داده‌ها به صورت طبیعی گروه‌های متمایزی را تشکیل می‌دهند یا به صورت تصادفی توزیع شده‌اند. دو روش رایج برای ارزیابی تمایل به خوشه‌بندی عبارتند از:

- **آمار هاپکینز (Hopkins Statistic):** آمار هاپکینز تمایل یک مجموعه داده به تشکیل خوشه‌ها را اندازه‌گیری می‌کند. این آمار توزیع فاصله بین نقاط به صورت تصادفی انتخاب شده و نقاط داده واقعی را مقایسه می‌کند. نمره پایین هاپکینز نشان‌دهنده احتمال بالاتر خوشه‌بندی است.
- **ضریب سیلوئت (Silhouette Score):** ضریب سیلوئت میزان جداسازی خوشه‌ها را کمی‌سازی می‌کند. این مقدار از -۱ تا ۱ متغیر است، که در آن یک مقدار مثبت بالا نشان‌دهنده خوشه‌های به خوبی تعریف شده است، نمره‌ای نزدیک به ۰ نشان‌دهنده خوشه‌های همپوشان است و مقادیر منفی به معنای خوشه‌بندی نادرست است.

این روش‌ها در تصمیم‌گیری برای ادامه تحلیل خوشه و اینکه کدام الگوریتم خوشه‌بندی برای مجموعه داده مناسب‌تر است، کمک می‌کنند.

کاربرد عملی در پایتون (Python) برای ارزیابی تمایل به خوشه‌بندی با استفاده از آمار هاپکینز:

بیایید یک مثال عملی با استفاده از آمار هاپکینز برای ارزیابی تمایل به خوشه‌بندی را در نظر بگیریم.

```
In [7]: from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
import numpy as np

def hopkins_statistic(X, m=None):
    n, d = X.shape
    if m is None:
        m = int(0.1 * n) # 10% of dataset

    # نمونه‌گیری تصادفی از داده
    random_indices = np.random.choice(n, m, replace=False)
    X_sample = X[random_indices]

    # در بازه‌ی داده (uniform random) نقاط مصنوعی
    min_vals, max_vals = np.min(X, axis=0), np.max(X, axis=0)
    synthetic_points = np.random.uniform(min_vals, max_vals, size=(m, d))

    # فاصله‌ی نقاط واقعی انتخاب‌شده تا نزدیک‌ترین همسایه واقعی (غیر از خودش)
    nn_real = NearestNeighbors(n_neighbors=2).fit(X)
    distances_real, _ = nn_real.kneighbors(X_sample)
    w = np.sum(distances_real[:, 1]) # 0 = فاصله با خودش

    # فاصله‌ی نقاط مصنوعی تا نزدیک‌ترین همسایه‌ی واقعی
    nn_synth = NearestNeighbors(n_neighbors=1).fit(X)
```

```
distances_synth, _ = nn_synth.kneighbors(synthetic_points)
u = np.sum(distances_synth)

# فرمول Hopkins
H = u / (u + w)
return H

# تولید داده‌ی تست
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)
X = StandardScaler().fit_transform(X)

# محاسبه Hopkins Statistic
hopkins_score = hopkins_statistic(X)
print(f"Hopkins Statistic: {hopkins_score:.4f}")
```

Hopkins Statistic: 0.9366

توضیحات مثال کد پایتون

در این مثال، شاخص **هاپکینز (Hopkins Statistic)** برای سنجش خوشه‌پذیری داده‌ها محاسبه شده است. این شاخص نشان می‌دهد که آیا داده‌ها به‌صورت تصادفی توزیع شده‌اند یا دارای ساختار خوشه‌ای هستند:

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند و سپس استانداردسازی شده‌اند.
- تابع `hopkins_statistic` با انتخاب تصادفی نمونه‌هایی از داده و تولید نقاط مصنوعی، فاصله‌ها را محاسبه می‌کند.
- شاخص هاپکینز با فرمول $H = u / (u + w)$ محاسبه می‌شود که در آن `u` مجموع فاصله‌های نقاط مصنوعی و `w` مجموع فاصله‌های نقاط واقعی است.
- مقدار این شاخص بین ۰ تا ۱ است. اگر مقدار نزدیک به ۱ باشد، داده‌ها خوشه‌پذیر هستند؛ اگر نزدیک به ۰.۵ باشد، داده‌ها تصادفی‌اند.

این شاخص ابزار مفیدی برای ارزیابی اولیه خوشه‌بندی است و قبل از اجرای الگوریتم‌هایی مانند K-Means یا DBSCAN می‌تواند به تصمیم‌گیری درباره مناسب بودن خوشه‌بندی کمک کند.

8.5.2. Determining the number of clusters

تعیین تعداد بهینه خوشه‌ها (Determining the optimal number of clusters)

تعیین تعداد بهینه خوشه‌ها یک جنبه حیاتی از تحلیل خوشه (cluster analysis) است. انواع مختلفی از روش‌ها برای این منظور به کار گرفته می‌شوند و برخی رویکردهای رایج عبارتند از:

- **روش آرنج (Elbow Method):** روش آرنج شامل رسم تغییرات توضیح داده شده (explained variation) به عنوان تابعی از تعداد خوشه‌ها و جستجو برای نقطه "آرنج" است. نقطه‌ای که در آن نرخ کاهش به شدت تغییر می‌کند، ممکن است تعداد بهینه خوشه‌ها را نشان دهد.
- **تحلیل سیلوئت (Silhouette Analysis):** تحلیل سیلوئت ارزیابی می‌کند که خوشه‌ها چقدر به خوبی از هم جدا شده‌اند. این روش یک نمره سیلوئت (silhouette score) را برای تعدادهای مختلف خوشه‌ها محاسبه می‌کند و نمره بالاتر نشان‌دهنده خوشه‌بندی بهتر تعریف شده است.
- **آمار گپ (Gap Statistics):** آمار گپ کیفیت خوشه‌بندی مجموعه داده را با کیفیت توزیع تصادفی مقایسه می‌کند. یک آمار گپ بزرگتر نشان‌دهنده تعداد مناسب‌تری از خوشه‌ها است.

انتخاب روش صحیح به ویژگی‌های مجموعه داده و نتیجه مطلوب از تحلیل بستگی دارد.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای تعیین تعداد خوشه‌ها با استفاده از روش آرنج:

بیایید یک مثال عملی از کاربرد عملی در پایتون (Python) برای تعیین تعداد خوشه‌ها با استفاده از روش آرنج (Elbow Method) را در نظر بگیریم.

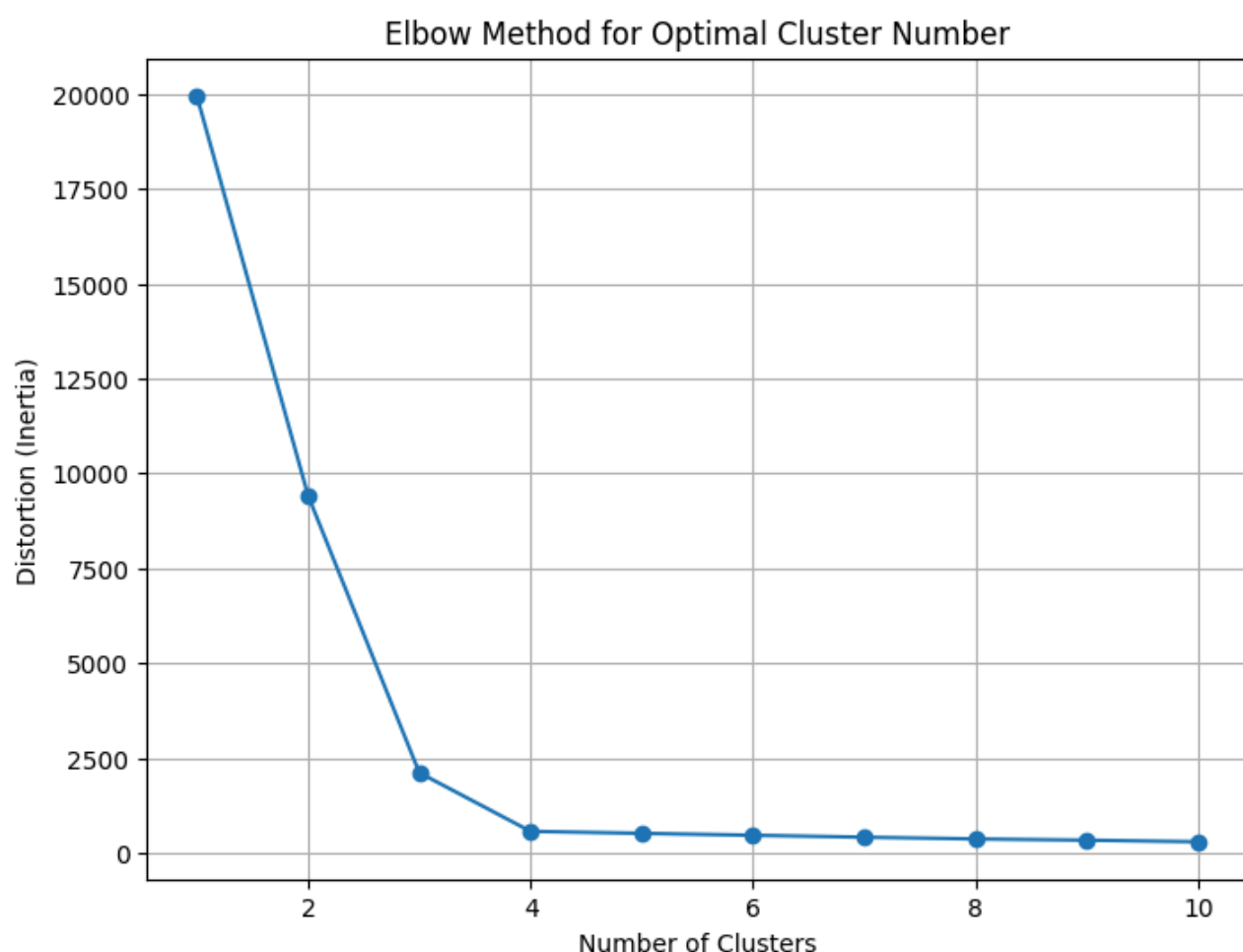
```
In [9]: import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import numpy as np

# --- برای تعداد مختلف خوشه‌ها Distortion تابع محاسبه معیار ---
def calculate_distortion(X, max_clusters):
    distortions = []
    for i in range(1, max_clusters + 1):
        kmeans = KMeans(n_clusters=i, random_state=42)
        kmeans.fit(X)
        distortions.append(kmeans.inertia_) # مجموع فاصله‌ها از مراکز خوشه‌ها
    return distortions

# --- تولید داده‌های مصنوعی با ۴ خوشه ---
X, _ = make_blobs(n_samples=300, centers=4, random_state=42, cluster_std=1.0)

# --- برای تعداد خوشه‌های مختلف Distortion محاسبه ---
max_clusters = 10
distortions = calculate_distortion(X, max_clusters)

# --- برای تعیین تعداد بهینه خوشه‌ها Elbow رسم نمودار روش ---
plt.figure(figsize=(8, 6))
plt.plot(range(1, max_clusters + 1), distortions, marker='o')
plt.title('Elbow Method for Optimal Cluster Number')
plt.xlabel('Number of Clusters')
plt.ylabel('Distortion (Inertia)')
plt.grid(True)
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از روش **Elbow** برای تعیین تعداد بهینه خوشه‌ها در الگوریتم **K-Means** استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مصنوعی با ۴ خوشه تولید شده‌اند.
 - برای تعداد خوشه‌های مختلف (از ۱ تا ۱۰)، الگوریتم K-Means اجرا شده و مقدار **Distortion** یا **Inertia** محاسبه شده است.
 - نمودار حاصل نشان می‌دهد که چگونه مقدار Distortion با افزایش تعداد خوشه‌ها کاهش می‌یابد.
 - نقطه‌ای که کاهش Distortion کند می‌شود (زاویه‌ی نمودار)، به‌عنوان **تعداد بهینه خوشه‌ها** در نظر گرفته می‌شود.
- روش Elbow یکی از ساده‌ترین و مؤثرترین روش‌ها برای انتخاب تعداد خوشه‌هاست و در بسیاری از مسائل خوشه‌بندی کاربرد دارد.

8.5.3. Measuring clustering quality: extrinsic methods

روش‌های بیرونی برای اندازه‌گیری کیفیت خوشه‌بندی (Extrinsic methods for measuring clustering quality)

روش‌های بیرونی برای اندازه‌گیری کیفیت خوشه‌بندی (clustering quality) شامل ارزیابی عملکرد الگوریتم‌های خوشه‌بندی (clustering algorithms) با استفاده از اطلاعات خارجی، مانند برچسب‌های واقعیت زمینی (ground truth labels) یا عضویت‌های کلاس شناخته شده، است. معیارهای بیرونی رایج عبارتند از:

- **Adjusted Rand Index (ARI): ARI** شباهت بین برچسب‌های واقعی و برچسب‌های اختصاص داده شده توسط الگوریتم خوشه‌بندی را اندازه‌گیری می‌کند. این معیار تمام جفت‌های نمونه را در نظر می‌گیرد و ارزیابی می‌کند که آیا آن‌ها در یک خوشه یا خوشه‌های متفاوت قرار گرفته‌اند و نمره‌ای بین -۱ و ۱ ارائه می‌دهد.
- **Normalized Mutual Information (NMI): NMI** اطلاعات متقابل (mutual information) بین برچسب‌های واقعی و برچسب‌های پیش‌بینی شده را کمی‌سازی می‌کند، که توسط آنتروپی (entropy) برچسب‌ها نرمال‌سازی شده است. مانند ARI، NMI از ۰ تا ۱ متغیر است، که مقادیر بالاتر نشان‌دهنده کیفیت خوشه‌بندی بهتر است.

این معیارها به ارزیابی اینکه یک الگوریتم خوشه‌بندی چقدر با ساختار شناخته شده داده‌ها هماهنگ است، کمک می‌کنند و بینشی را در مورد اثربخشی آن در ثبت الگوهای معنادار فراهم می‌سازند.

مثال عملی در پایتون (Python) برای اندازه‌گیری کیفیت خوشه‌بندی با استفاده از Adjusted Rand Index:

```
In [11]: from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import adjusted_rand_score
import numpy as np

# --- تولید داده‌های مصنوعی با برچسب‌های واقعی ---
X, true_labels = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# --- اعمال الگوریتم خوشه‌بندی K-Means ---
kmeans = KMeans(n_clusters=3, random_state=42)
predicted_labels = kmeans.fit_predict(X)

# --- برای ارزیابی کیفیت خوشه‌بندی Adjusted Rand Index محاسبه شاخص ---
ari_score = adjusted_rand_score(true_labels, predicted_labels)

# --- نمایش نتیجه ---
print(f"Adjusted Rand Index: {ari_score}")
```

Adjusted Rand Index: 1.0

توضیحات مثال کد پایتون

در این مثال، از شاخص **Adjusted Rand Index (ARI)** برای ارزیابی کیفیت خوشه‌بندی استفاده شده است. این شاخص میزان تطابق بین برچسب‌های واقعی و برچسب‌های پیش‌بینی‌شده توسط الگوریتم خوشه‌بندی را اندازه‌گیری می‌کند:

- ابتدا داده‌های مصنوعی با سه خوشه و برچسب‌های واقعی تولید شده‌اند.
 - الگوریتم **K-Means** روی داده‌ها اجرا شده و برچسب‌های خوشه‌ای پیش‌بینی شده‌اند.
 - شاخص ARI با مقایسه برچسب‌های واقعی و پیش‌بینی‌شده محاسبه شده است.
 - مقدار این شاخص بین -1 تا 1 است. مقدار 1 نشان‌دهنده تطابق کامل، مقدار 0 نشان‌دهنده تطابق تصادفی، و مقدار منفی نشان‌دهنده تطابق ضعیف‌تر از تصادف است.
- این شاخص ابزار قدرتمندی برای ارزیابی عملکرد الگوریتم‌های خوشه‌بندی در شرایطی است که برچسب‌های واقعی در دسترس باشند.

8.5.4. Intrinsic methods

روش‌های درونی برای اندازه‌گیری کیفیت خوشه‌بندی (Intrinsic methods for measuring clustering quality)

روش‌های درونی برای اندازه‌گیری کیفیت خوشه‌بندی (clustering quality) عملکرد الگوریتم‌های خوشه‌بندی (clustering algorithms) را تنها بر اساس ساختار داخلی داده‌ها و بدون اتکا به اطلاعات خارجی ارزیابی می‌کنند. معیارهای درونی رایج عبارتند از:

- **Silhouette Score:** Silhouette Score میزان جداسازی خوشه‌ها را کمی‌سازی می‌کند. این مقدار برای هر نقطه داده‌ای محاسبه شده و در بازه -۱ تا ۱ قرار دارد. یک نمره Silhouette Score بالاتر نشان‌دهنده خوشه‌های بهتر تعریف شده است.
- **Davies-Bouldin Index:** Davies-Bouldin Index فشردگی و جداسازی بین خوشه‌ها را اندازه‌گیری می‌کند. یک Davies-Bouldin Index پایین‌تر نشان‌دهنده خوشه‌بندی بهتر است، با حداقل فاصله درون‌خوشه‌ای (intra-cluster distance) و حداکثر فاصله بین‌خوشه‌ای (inter-cluster distance).
- **Calinski-Harabasz Index:** Calinski-Harabasz Index نسبت واریانس بین‌خوشه‌ای (between-cluster variance) به واریانس درون‌خوشه‌ای (within-cluster variance) را ارزیابی می‌کند. یک شاخص بالاتر نشان‌دهنده خوشه‌های بهتر تعریف شده است.

این معیارها بدون نیاز به اعتبارسنجی خارجی، بینش‌هایی را در مورد کیفیت ذاتی خوشه‌های تشکیل شده توسط الگوریتم فراهم می‌کنند.

مثال دنیای واقعی از کاربرد عملی در پایتون (Python) برای اندازه‌گیری کیفیت خوشه‌بندی با استفاده از Silhouette Score:

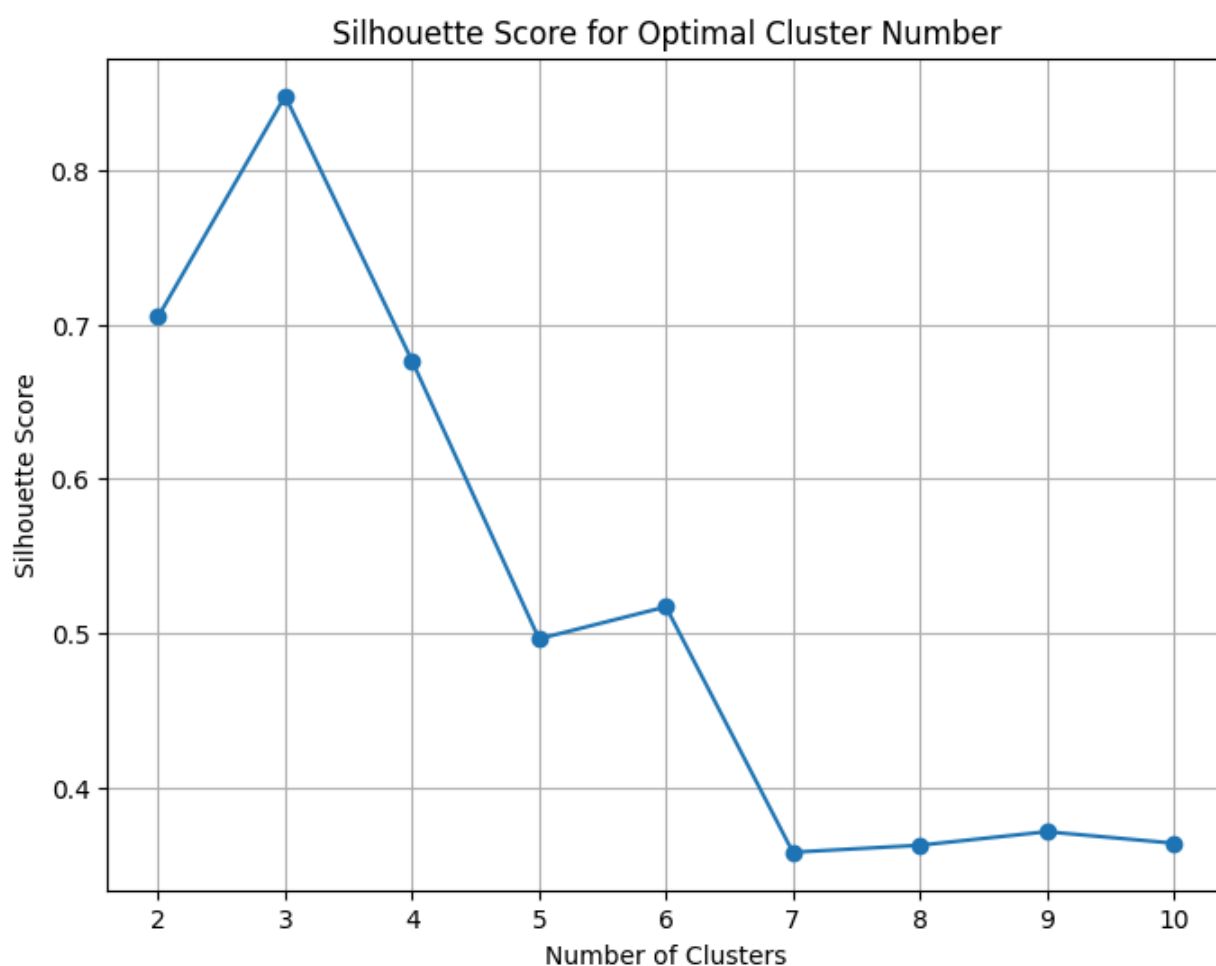
```
In [13]: from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
import numpy as np
```

```
# --- تولید داده‌های مصنوعی با ۳ خوشه ---
X, _ = make_blobs(n_samples=300, centers=3, random_state=42, cluster_std=1.0)

# --- Silhouette Score با تعداد خوشه‌های مختلف و محاسبه K-Means اجرای ---
cluster_range = range(2, 11)
silhouette_scores = []

for n_clusters in cluster_range:
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    silhouette_scores.append(score)

# --- برای تحلیل تعداد بهینه خوشه‌ها Silhouette رسم نمودار ---
plt.figure(figsize=(8, 6))
plt.plot(cluster_range, silhouette_scores, marker='o')
plt.title('Silhouette Score for Optimal Cluster Number')
plt.xlabel('Number of Clusters')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.show()
```



توضیحات مثال کد پایتون

در این مثال، از معیار **Silhouette Score** برای تعیین تعداد بهینه خوشه‌ها در الگوریتم **K-Means** استفاده شده است. این معیار میزان جدایی بین خوشه‌ها و انسجام درون خوشه‌ها را اندازه‌گیری می‌کند:

- ابتدا داده‌های مصنوعی با سه خوشه تولید شده‌اند.
- الگوریتم K-Means برای تعداد خوشه‌های مختلف (۲ تا ۱۰) اجرا شده است.
- برای هر تعداد خوشه، مقدار Silhouette Score محاسبه شده که بین ۱- تا ۱ است. مقدار بالاتر نشان‌دهنده خوشه‌بندی بهتر است.
- نمودار حاصل نشان می‌دهد که کدام تعداد خوشه منجر به بالاترین Silhouette Score شده و بنابراین بهینه‌ترین انتخاب است.

این روش مکملی عالی برای روش Elbow است و در بسیاری از پروژه‌های خوشه‌بندی کاربرد دارد، به‌ویژه زمانی که برچسب‌های واقعی در دسترس نیستند.

The End of Chapter 8